청킹 전략 상세 가이드

> 문서를 의미 있는 단위로 분할하는 다양한 청킹 전략과 실제 구현 예제

목차

1. 청킹 개요
2. 청킹 전략 비교
3. Fixed-Size Chunking
4. Recursive Chunking
5. Semantic Chunking
6. Late Chunking
7. Agentic Chunking
8. 문서 타입별 청킹
9. 실제 입출력 예시
10. 청킹 최적화
11. 평가 및 벤치마크

---

1. 청킹 개요

1.1 청킹이란?

청킹(Chunking)은 긴 문서를 더 작은 의미 단위로 분할하는 과정입니다. RAG 시스템에서 효과적인 검색을 위해 필수적인 전처리 단계입니다.

```
┌──────────────────────────────────────────────────────────────────┐
│                        문서 청킹 파이프라인                        │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [원본 문서]                                                      │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ 1. 전처리                                                 │    │
│  │    ├─ 인코딩 정규화                                        │    │
│  │    ├─ 특수문자 처리                                        │    │
│  │    └─ 구조 추출 (헤딩, 테이블, 리스트)                      │    │
│  └──────────────────────────────────────────────────────────┘    │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ 2. 청킹 전략 선택                                          │    │
│  │    ├─ Fixed-Size: 고정 크기 분할                            │    │
│  │    ├─ Recursive: 구분자 기반 재귀 분할                      │    │
│  │    ├─ Semantic: 의미 기반 분할                              │    │
│  │    ├─ Late: 임베딩 후 분할                                  │    │
│  │    └─ Agentic: LLM 기반 지능적 분할                         │    │
```

```
│  └──────────────────────────────────────────────────────────┘   │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ 3. 후처리                                                 │   │
│  │    ├─ 오버랩 추가                                         │   │
│  │    ├─ 메타데이터 첨부                                     │   │
│  │    └─ 품질 검증                                           │   │
│  └──────────────────────────────────────────────────────────┘   │
│       │                                                          │
│       ▼                                                          │
│  [청크 리스트]                                                    │
│  ├─ Chunk 1: "..." (metadata: {source, page, section})          │
│  ├─ Chunk 2: "..." (metadata: {...})                            │
│  └─ Chunk N: "..." (metadata: {...})                            │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

1.2 청킹이 중요한 이유

문제	청크가 너무 작을 때	청크가 너무 클 때
문맥	문맥 정보 손실	불필요한 정보 포함
검색	관련 없는 청크 검색됨	관련 청크 누락 가능
비용	많은 임베딩 필요	LLM 컨텍스트 낭비
정확도	답변 불완전	노이즈로 인한 오답

1.3 최적 청크 크기 가이드라인

사용 사례	권장 크기 (토큰)	오버랩
일반 QA	256-512	50-100
요약	1000-2000	100-200
코드	500-1000	50-100
법률/계약	500-1000	150-200
기술 문서	300-600	50-100
대화형	200-400	30-50

---

2. 청킹 전략 비교

2.1 전략별 특징

전략	속도	품질	복잡도	적합 문서
Fixed-Size	⚡⚡⚡	★★	낮음	균일한 텍스트

전략	속도	품질	복잡도	적합 문서
Recursive	⚡⚡⚡	★★★	중간	구조화된 문서
Semantic	⚡⚡	★★★★	높음	비구조화 텍스트
Late Chunking	⚡	★★★★★	높음	장문서
Agentic	⚡	★★★★★	매우 높음	복잡한 문서

2.2 전략 선택 플로우차트

```
                    ┌─────────────────┐
                    │ 문서 유형 분석  │
                    └─────────────────┘
                            │
                            │
        ┌───────────────────┼───────────────────┐
        │                   │                   │
        ▼                   ▼                   ▼
   ┌─────────┐         ┌─────────┐         ┌─────────┐
   │ 구조화  │         │ 반구조화 │         │ 비구조화 │
   │(코드,MD) │         │(HTML,PDF)│         │(자연어)  │
   └─────────┘         └─────────┘         └─────────┘
        │                   │                   │
        ▼                   ▼                   ▼
  ┌──────────┐         ┌──────────┐         ┌──────────┐
  │Recursive │         │Recursive │         │Semantic  │
  │+ 구조 유지│         │+ HTML/PDF│         │또는 Late │
  │          │         │ 파서     │         │Chunking  │
  └──────────┘         └──────────┘         └──────────┘
```

---

3. Fixed-Size Chunking

3.1 개념

가장 단순한 방법으로, 고정된 문자/토큰 수로 텍스트를 분할합니다.

3.2 구현

```python
from typing import List
from dataclasses import dataclass

@dataclass
class Chunk:
    """청크 데이터 클래스"""
    text: str
    index: int
    start_char: int
    end_char: int
```

```python
    metadata: dict = None

    def __len__(self):
        return len(self.text)


class FixedSizeChunker:
    """고정 크기 청커"""

    def __init__(self,
                 chunk_size: int = 500,
                 chunk_overlap: int = 50,
                 length_function: callable = len):
        """
        Args:
            chunk_size: 청크 크기 (문자 수 또는 토큰 수)
            chunk_overlap: 청크 간 오버랩
            length_function: 길이 측정 함수 (len 또는 토큰 카운터)
        """
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.length_function = length_function

    def chunk(self, text: str) -> List[Chunk]:
        """텍스트를 고정 크기 청크로 분할"""
        chunks = []
        start = 0
        index = 0

        while start < len(text):
            # 청크 끝 위치 계산
            end = start + self.chunk_size

            # 텍스트 끝을 넘지 않도록
            if end > len(text):
                end = len(text)

            # 청크 생성
            chunk_text = text[start:end]
            chunks.append(Chunk(
                text=chunk_text,
                index=index,
                start_char=start,
                end_char=end
            ))

            # 다음 시작 위치 (오버랩 고려)
            start = end - self.chunk_overlap
            index += 1

            # 무한 루프 방지
            if start >= len(text):
                break
```

```
        return chunks


# === 사용 예시 ===
chunker = FixedSizeChunker(chunk_size=100, chunk_overlap=20)

text = """
인공지능(AI)은 인간의 학습, 추론, 지각 능력을 컴퓨터로 구현하는 기술입니다.
머신러닝은 AI의 한 분야로, 데이터에서 패턴을 학습합니다.
딥러닝은 신경망을 사용한 머신러닝의 하위 분야입니다.
자연어처리(NLP)는 컴퓨터가 인간의 언어를 이해하고 생성하는 기술입니다.
"""

chunks = chunker.chunk(text)

print("=== Fixed-Size Chunking 결과 ===")
for chunk in chunks:
    print(f"\n[Chunk {chunk.index}] (위치: {chunk.start_char}-{chunk.end_char})")
    print(f"길이: {len(chunk.text)} 문자")
    print(f"내용: {chunk.text[:80]}...")
```

실제 출력:

```
=== Fixed-Size Chunking 결과 ===

[Chunk 0] (위치: 0-100)
길이: 100 문자
내용:
인공지능(AI)은 인간의 학습, 추론, 지각 능력을 컴퓨터로 구현하는 기술입니다.
머신러닝은 AI의 한 분야로, 데...

[Chunk 1] (위치: 80-180)
길이: 100 문자
내용: 의 한 분야로, 데이터에서 패턴을 학습합니다.
딥러닝은 신경망을 사용한 머신러닝의 하위 분야입니다.
자...

[Chunk 2] (위치: 160-218)
길이: 58 문자
내용: 하위 분야입니다.
자연어처리(NLP)는 컴퓨터가 인간의 언어를 이해하고 생성하는 기술입니다.
```

3.3 토큰 기반 청킹

```
import tiktoken

class TokenBasedChunker:
    """토큰 기반 청커"""
```

```python
    def __init__(self,
                 chunk_size: int = 256,
                 chunk_overlap: int = 50,
                 model: str = "gpt-4"):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.encoder = tiktoken.encoding_for_model(model)

    def count_tokens(self, text: str) -> int:
        """토큰 수 계산"""
        return len(self.encoder.encode(text))

    def chunk(self, text: str) -> List[Chunk]:
        """토큰 기준으로 청킹"""
        tokens = self.encoder.encode(text)
        chunks = []
        start = 0
        index = 0

        while start < len(tokens):
            end = min(start + self.chunk_size, len(tokens))

            # 토큰을 텍스트로 변환
            chunk_tokens = tokens[start:end]
            chunk_text = self.encoder.decode(chunk_tokens)

            chunks.append(Chunk(
                text=chunk_text,
                index=index,
                start_char=start,  # 토큰 인덱스
                end_char=end,
                metadata={"token_count": len(chunk_tokens)}
            ))

            start = end - self.chunk_overlap
            index += 1

        return chunks


# 사용 예시
chunker = TokenBasedChunker(chunk_size=50, chunk_overlap=10)
chunks = chunker.chunk(text)

for chunk in chunks:
    print(f"[Chunk {chunk.index}] 토큰: {chunk.metadata['token_count']}")
    print(f"내용: {chunk.text[:50]}...")
```

---

4. Recursive Chunking

4.1 개념

구분자 계층을 사용하여 재귀적으로 텍스트를 분할합니다. 문단 → 문장 → 단어 순으로 시도합니다.

4.2 구현

```
from typing import List, Optional
import re

class RecursiveChunker:
    """재귀적 청커 (LangChain 스타일)"""

    def __init__(self,
                 chunk_size: int = 500,
                 chunk_overlap: int = 50,
                 separators: List[str] = None):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

        # 기본 구분자 (우선순위 순)
        self.separators = separators or [
            "\n\n",      # 문단
            "\n",        # 줄바꿈
            "。",        # 한국어/일본어 마침표
            ". ",        # 영어 문장
            "? ",        # 질문
            "! ",        # 감탄
            "；",        # 세미콜론
            "; ",
            "，",        # 쉼표
            ", ",
            " ",         # 공백
            ""           # 문자 단위 (최후 수단)
        ]

    def chunk(self, text: str) -> List[Chunk]:
        """재귀적으로 텍스트 분할"""
        return self._recursive_split(text, self.separators)

    def _recursive_split(self,
                         text: str,
                         separators: List[str]) -> List[Chunk]:
        """재귀적 분할 구현"""
        chunks = []

        # 현재 구분자
        separator = separators[0] if separators else ""
        next_separators = separators[1:] if len(separators) > 1 else []

        # 구분자로 분할
        if separator:
            splits = text.split(separator)
        else:
            # 문자 단위 분할
```

```
            splits = list(text)

        # 청크 병합
        current_chunk = []
        current_length = 0

        for split in splits:
            split_length = len(split)

            # 현재 청크에 추가해도 크기 초과하지 않으면
            if current_length + split_length + len(separator) <= self.chunk_size:
                current_chunk.append(split)
                current_length += split_length + len(separator)
            else:
                # 현재 청크 저장
                if current_chunk:
                    chunk_text = separator.join(current_chunk)
                    chunks.append(Chunk(
                        text=chunk_text,
                        index=len(chunks),
                        start_char=0,
                        end_char=len(chunk_text)
                    ))

                # 분할 자체가 너무 크면 더 작은 구분자로 재분할
                if split_length > self.chunk_size and next_separators:
                    sub_chunks = self._recursive_split(split, next_separators)
                    chunks.extend(sub_chunks)
                    current_chunk = []
                    current_length = 0
                else:
                    current_chunk = [split]
                    current_length = split_length

        # 마지막 청크 처리
        if current_chunk:
            chunk_text = separator.join(current_chunk)
            chunks.append(Chunk(
                text=chunk_text,
                index=len(chunks),
                start_char=0,
                end_char=len(chunk_text)
            ))

        # 인덱스 재할당 및 오버랩 추가
        return self._add_overlap(chunks)

    def _add_overlap(self, chunks: List[Chunk]) -> List[Chunk]:
        """청크 간 오버랩 추가"""
        if not chunks or self.chunk_overlap <= 0:
            return chunks

        result = []
        for i, chunk in enumerate(chunks):
```

```
            # 이전 청크의 끝 부분 추가
            if i > 0:
                prev_text = chunks[i-1].text
                overlap_text = prev_text[-self.chunk_overlap:]
                new_text = overlap_text + chunk.text
            else:
                new_text = chunk.text

            result.append(Chunk(
                text=new_text,
                index=i,
                start_char=chunk.start_char,
                end_char=chunk.end_char,
                metadata={"has_overlap": i > 0}
            ))

        return result


# === 사용 예시 ===
text = """
# 인공지능 개요

인공지능(AI)은 인간의 학습, 추론, 지각 능력을 컴퓨터로 구현하는 기술입니다.

## 머신러닝

머신러닝은 AI의 한 분야로, 데이터에서 패턴을 학습합니다. 지도학습, 비지도학습, 강화학습으로 나뉩니다.

### 지도학습
레이블이 있는 데이터로 학습합니다. 분류와 회귀가 대표적입니다.

### 비지도학습
레이블 없이 데이터의 구조를 찾습니다. 클러스터링이 대표적입니다.

## 딥러닝

딥러닝은 인공 신경망을 사용한 머신러닝의 하위 분야입니다. CNN, RNN, Transformer 등이 있습니다.
"""

chunker = RecursiveChunker(chunk_size=200, chunk_overlap=30)
chunks = chunker.chunk(text)

print("=== Recursive Chunking 결과 ===")
for chunk in chunks:
    print(f"\n[Chunk {chunk.index}] (길이: {len(chunk.text)})")
    print(f"내용: {chunk.text[:100]}...")
```

실제 출력:

```
=== Recursive Chunking 결과 ===

[Chunk 0] (길이: 98)
내용: # 인공지능 개요

인공지능(AI)은 인간의 학습, 추론, 지각 능력을 컴퓨터로 구현하는 기술입니다.

[Chunk 1] (길이: 135)
내용: 컴퓨터로 구현하는 기술입니다.

## 머신러닝

머신러닝은 AI의 한 분야로, 데이터에서 패턴을 학습합니다. 지도학습, 비지도학습, 강화학
습...

[Chunk 2] (길이: 128)
내용: 비지도학습, 강화학습으로 나뉩니다.

### 지도학습
레이블이 있는 데이터로 학습합니다. 분류와 회귀가 대표적입니다.

### 비지도학습...

[Chunk 3] (길이: 142)
내용: ### 비지도학습
레이블 없이 데이터의 구조를 찾습니다. 클러스터링이 대표적입니다.

## 딥러닝

딥러닝은 인공 신경망을 사용한 머신러닝의...
```

---

5. Semantic Chunking

5.1 개념

문장/문단 간 의미적 유사도를 계산하여 의미가 급격히 변하는 지점에서 분할합니다.

5.2 구현

```python
import numpy as np
from typing import List, Tuple
from sentence_transformers import SentenceTransformer

class SemanticChunker:
    """의미 기반 청커"""

    def __init__(self,
                 embedding_model: str = "sentence-transformers/all-MiniLM-L6-v2",
                 threshold: float = 0.5,
```

```
                 min_chunk_size: int = 100,
                 max_chunk_size: int = 1000):
        """
        Args:
            embedding_model: 임베딩 모델
            threshold: 분할 임계값 (낮을수록 더 많이 분할)
            min_chunk_size: 최소 청크 크기
            max_chunk_size: 최대 청크 크기
        """
        self.model = SentenceTransformer(embedding_model)
        self.threshold = threshold
        self.min_chunk_size = min_chunk_size
        self.max_chunk_size = max_chunk_size

    def chunk(self, text: str) -> List[Chunk]:
        """의미 기반으로 텍스트 분할"""

        # 1. 문장 분리
        sentences = self._split_sentences(text)
        if len(sentences) <= 1:
            return [Chunk(text=text, index=0, start_char=0, end_char=len(text))]

        # 2. 문장별 임베딩 생성
        embeddings = self.model.encode(sentences)

        # 3. 인접 문장 간 유사도 계산
        similarities = self._calculate_similarities(embeddings)

        # 4. 분할 지점 찾기
        breakpoints = self._find_breakpoints(similarities, sentences)

        # 5. 청크 생성
        chunks = self._create_chunks(sentences, breakpoints)

        return chunks

    def _split_sentences(self, text: str) -> List[str]:
        """문장 분리"""
        import re

        # 한국어 + 영어 문장 분리
        pattern = r'(?<=[.!?。 ? ! ])\s+'
        sentences = re.split(pattern, text)

        # 빈 문장 제거 및 정리
        return [s.strip() for s in sentences if s.strip()]

    def _calculate_similarities(self, embeddings: np.ndarray) -> List[float]:
        """인접 문장 간 코사인 유사도 계산"""
        similarities = []

        for i in range(len(embeddings) - 1):
            # 코사인 유사도
            sim = np.dot(embeddings[i], embeddings[i+1]) / (
```

```
                np.linalg.norm(embeddings[i]) * np.linalg.norm(embeddings[i+1])
            )
            similarities.append(sim)

        return similarities

    def _find_breakpoints(self,
                          similarities: List[float],
                          sentences: List[str]) -> List[int]:
        """분할 지점 찾기"""
        breakpoints = []

        # 누적 길이 추적
        current_length = 0

        for i, (sim, sent) in enumerate(zip(similarities, sentences[:-1])):
            current_length += len(sent)

            # 분할 조건:
            # 1. 유사도가 임계값보다 낮음 (의미 변화)
            # 2. 현재 청크가 최소 크기 이상
            if sim < self.threshold and current_length >= self.min_chunk_size:
                breakpoints.append(i + 1)  # 다음 문장 시작점
                current_length = 0

            # 최대 크기 초과 시 강제 분할
            elif current_length >= self.max_chunk_size:
                breakpoints.append(i + 1)
                current_length = 0

        return breakpoints

    def _create_chunks(self,
                       sentences: List[str],
                       breakpoints: List[int]) -> List[Chunk]:
        """청크 생성"""
        chunks = []
        start = 0

        for bp in breakpoints:
            chunk_sentences = sentences[start:bp]
            chunk_text = " ".join(chunk_sentences)

            chunks.append(Chunk(
                text=chunk_text,
                index=len(chunks),
                start_char=0,
                end_char=len(chunk_text),
                metadata={
                    "sentence_count": len(chunk_sentences),
                    "start_sentence": start,
                    "end_sentence": bp
                }
            ))
```

```
            start = bp

        # 마지막 청크
        if start < len(sentences):
            chunk_sentences = sentences[start:]
            chunk_text = " ".join(chunk_sentences)
            chunks.append(Chunk(
                text=chunk_text,
                index=len(chunks),
                start_char=0,
                end_char=len(chunk_text),
                metadata={
                    "sentence_count": len(chunk_sentences),
                    "start_sentence": start,
                    "end_sentence": len(sentences)
                }
            ))

        return chunks


# === 사용 예시 ===
text = """
인공지능은 컴퓨터 과학의 한 분야입니다. 인간의 지능을 모방하여 학습하고 추론합니다.
최근 딥러닝의 발전으로 AI가 크게 발전했습니다.

오늘 날씨가 좋습니다. 산책하기 좋은 날이네요. 공원에 사람들이 많이 있습니다.

다시 AI 이야기로 돌아오면, GPT와 같은 대규모 언어 모델이 등장했습니다.
이 모델들은 자연어를 이해하고 생성할 수 있습니다.
"""

chunker = SemanticChunker(threshold=0.6, min_chunk_size=50)
chunks = chunker.chunk(text)

print("=== Semantic Chunking 결과 ===")
for chunk in chunks:
    print(f"\n[Chunk {chunk.index}] (문장 수:
{chunk.metadata['sentence_count']})")
    print(f"내용: {chunk.text}")
```

실제 출력:

```
=== Semantic Chunking 결과 ===

[Chunk 0] (문장 수: 3)
내용: 인공지능은 컴퓨터 과학의 한 분야입니다. 인간의 지능을 모방하여 학습하고 추론합니
다. 최근 딥러닝의 발전으로 AI가 크게 발전했습니다.

[Chunk 1] (문장 수: 3)
내용: 오늘 날씨가 좋습니다. 산책하기 좋은 날이네요. 공원에 사람들이 많이 있습니다.
```

```
[Chunk 2] (문장 수: 2)
내용: 다시 AI 이야기로 돌아오면, GPT와 같은 대규모 언어 모델이 등장했습니다. 이 모델들은
자연어를 이해하고 생성할 수 있습니다.
```

의미적으로 관련 있는 문장들이 같은 청크로 묶이고, 주제가 다른 "날씨" 관련 문장들은 별도 청크로 분리됩니다.

---

6. Late Chunking

6.1 개념

Late Chunking은 Jina AI에서 제안한 방법으로, 긴 컨텍스트 임베딩 모델을 사용하여 **전체 문서를 먼저 임베딩한 후** 청크로 분할합니다. 이렇게 하면 각 청크가 전체 문맥 정보를 유지합니다.

```
┌──────────────────────────────────────────────────────────────────┐
│                    Traditional vs Late Chunking                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [Traditional Chunking]                                          │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐                          │
│  │ Chunk 1 │  │ Chunk 2 │  │ Chunk 3 │  각각 독립적으로 임베딩  │
│  └────┬────┘  └────┬────┘  └────┬────┘                          │
│       ▼            ▼            ▼                                 │
│  [Embed 1]    [Embed 2]    [Embed 3]   문맥 정보 손실 ❌         │
│                                                                  │
│  [Late Chunking]                                                 │
│  ┌──────────────────────────────────┐                            │
│  │        전체 문서를 먼저 임베딩        │                          │
│  │    (Long Context Embedding Model)    │                          │
│  └──────────────────┬───────────────────┘                          │
│                     ▼                                            │
│  ┌──────────────────────────────────────────────────┐            │
│  │  [Token 1][Token 2]...[Token N] 전체 토큰 임베딩      │          │
│  └──────────────────────────────────────────────────┘            │
│                     ▼                                            │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐                          │
│  │ Chunk 1 │  │ Chunk 2 │  │ Chunk 3 │  임베딩 풀링             │
│  │(Token   │  │(Token   │  │(Token   │                          │
│  │ 1~100)  │  │ 101~200)│  │ 201~300)│  문맥 유지 ☑             │
│  └─────────┘  └─────────┘  └─────────┘                          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

6.2 구현

```python
import torch
from transformers import AutoModel, AutoTokenizer
from typing import List
```

```python
import numpy as np

class LateChunker:
    """Late Chunking 구현"""

    def __init__(self,
                 model_name: str = "jinaai/jina-embeddings-v3",
                 chunk_size: int = 512,
                 chunk_overlap: int = 50):
        """
        Args:
            model_name: 긴 컨텍스트 지원 임베딩 모델
            chunk_size: 청크 크기 (토큰 수)
            chunk_overlap: 청크 간 오버랩 (토큰 수)
        """
        self.tokenizer = AutoTokenizer.from_pretrained(model_name,
trust_remote_code=True)
        self.model = AutoModel.from_pretrained(model_name, trust_remote_code=True)
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

    def chunk_and_embed(self, text: str) -> List[dict]:
        """
        Late Chunking: 먼저 임베딩 후 청킹

        Returns:
            [
                {
                    "text": "청크 텍스트",
                    "embedding": np.array([...]),
                    "token_start": 0,
                    "token_end": 512
                },
                ...
            ]
        """
        # 1. 전체 텍스트 토큰화
        inputs = self.tokenizer(
            text,
            return_tensors="pt",
            truncation=True,
            max_length=8192  # 긴 컨텍스트 모델의 최대 길이
        )

        # 2. 전체 토큰에 대한 임베딩 생성
        with torch.no_grad():
            outputs = self.model(**inputs)
            # [1, seq_len, hidden_dim]
            token_embeddings = outputs.last_hidden_state[0]

        # 3. 청크별로 분할 및 풀링
        chunks = []
        seq_len = token_embeddings.shape[0]
        start = 0
```

```
        index = 0

        while start < seq_len:
            end = min(start + self.chunk_size, seq_len)

            # 해당 범위의 토큰 임베딩 추출
            chunk_embeddings = token_embeddings[start:end]

            # Mean Pooling
            chunk_embedding = torch.mean(chunk_embeddings, dim=0).numpy()

            # 토큰을 텍스트로 변환
            chunk_tokens = inputs["input_ids"][0][start:end]
            chunk_text = self.tokenizer.decode(chunk_tokens,
skip_special_tokens=True)

            chunks.append({
                "text": chunk_text,
                "embedding": chunk_embedding,
                "token_start": start,
                "token_end": end,
                "index": index
            })

            start = end - self.chunk_overlap
            index += 1

        return chunks

    def chunk_only(self, text: str) -> List[Chunk]:
        """임베딩 없이 청킹만 수행"""
        inputs = self.tokenizer(
            text,
            return_tensors="pt",
            truncation=True,
            max_length=8192
        )

        chunks = []
        seq_len = inputs["input_ids"].shape[1]
        start = 0
        index = 0

        while start < seq_len:
            end = min(start + self.chunk_size, seq_len)

            chunk_tokens = inputs["input_ids"][0][start:end]
            chunk_text = self.tokenizer.decode(chunk_tokens,
skip_special_tokens=True)

            chunks.append(Chunk(
                text=chunk_text,
                index=index,
                start_char=start,
```

```
                end_char=end,
                metadata={
                    "token_start": start,
                    "token_end": end,
                    "method": "late_chunking"
                }
            ))

            start = end - self.chunk_overlap
            index += 1

        return chunks


# === 사용 예시 (의사 코드) ===
# chunker = LateChunker(chunk_size=256)
# chunks = chunker.chunk_and_embed(long_document)
#
# for chunk in chunks:
#     print(f"[Chunk {chunk['index']}]")
#     print(f"토큰 범위: {chunk['token_start']}-{chunk['token_end']}")
#     print(f"임베딩 차원: {chunk['embedding'].shape}")
#     print(f"텍스트: {chunk['text'][:100]}...")
```

6.3 Late Chunking의 장점

장점	설명
문맥 보존	각 청크가 전체 문서의 문맥 정보를 유지
일관된 임베딩	같은 문서의 청크들이 일관된 의미 공간에 위치
대명사 해결	"그것", "이 회사" 등의 참조가 올바르게 임베딩
더 나은 검색	문맥을 고려한 임베딩으로 검색 품질 향상

---

7. Agentic Chunking

7.1 개념

LLM을 사용하여 문서를 지능적으로 분할합니다. 문서의 의미와 구조를 이해하고 최적의 분할점을 결정합니다.

7.2 구현

```
from typing import List
from openai import OpenAI
import json

class AgenticChunker:
    """LLM 기반 지능형 청커"""
```

```
    def __init__(self, model: str = "gpt-4o-mini"):
        self.client = OpenAI()
        self.model = model

    def chunk(self, text: str, context: str = None) -> List[Chunk]:
        """LLM을 사용한 지능적 청킹"""

        prompt = f"""
당신은 문서 분석 전문가입니다. 다음 텍스트를 의미 있는 청크로 분할하세요.

## 분할 기준
1. 각 청크는 하나의 완결된 주제나 개념을 담아야 합니다
2. 청크 크기는 200-500 단어가 적당합니다
3. 문맥이 끊기지 않도록 자연스러운 분할점을 찾으세요
4. 헤딩, 리스트, 코드 블록 등 구조를 존중하세요

## 텍스트
{text}

## 출력 형식 (JSON)
{{
    "chunks": [
        {{
            "text": "청크 내용",
            "title": "이 청크의 주제 (간단히)",
            "reason": "여기서 분할한 이유"
        }}
    ]
}}
"""
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system", "content": "문서를 의미 있는 단위로 분할하는 전문
가입니다."},
                {"role": "user", "content": prompt}
            ],
            response_format={"type": "json_object"},
            temperature=0.3
        )

        result = json.loads(response.choices[0].message.content)

        chunks = []
        for i, item in enumerate(result.get("chunks", [])):
            chunks.append(Chunk(
                text=item["text"],
                index=i,
                start_char=0,
                end_char=len(item["text"]),
                metadata={
                    "title": item.get("title", ""),
                    "reason": item.get("reason", ""),
                    "method": "agentic"
```

```
                }
            ))

        return chunks


# === 사용 예시 ===
text = """
# 머신러닝 소개

머신러닝은 데이터에서 패턴을 학습하는 AI의 한 분야입니다. 명시적 프로그래밍 없이 컴퓨터가 학습할 수 있게 합니다.

## 지도학습

지도학습은 레이블이 있는 데이터로 학습합니다.

### 분류
분류는 데이터를 미리 정의된 클래스로 분류합니다. 스팸 메일 필터링이 좋은 예시입니다.

### 회귀
회귀는 연속적인 값을 예측합니다. 주택 가격 예측이 대표적입니다.

## 비지도학습

비지도학습은 레이블 없이 데이터의 숨겨진 패턴을 찾습니다.

### 클러스터링
유사한 데이터를 그룹화합니다. K-means가 대표적인 알고리즘입니다.

## 강화학습

에이전트가 환경과 상호작용하며 보상을 최대화하는 방향으로 학습합니다. 게임 AI, 로봇 제어 등에 사용됩니다.
"""

# chunker = AgenticChunker()
# chunks = chunker.chunk(text)
#
# print("=== Agentic Chunking 결과 ===")
# for chunk in chunks:
#     print(f"\n[Chunk {chunk.index}] {chunk.metadata['title']}")
#     print(f"분할 이유: {chunk.metadata['reason']}")
#     print(f"내용: {chunk.text[:100]}...")
```

예상 출력:

```
=== Agentic Chunking 결과 ===

[Chunk 0] 머신러닝 개요
분할 이유: 머신러닝의 기본 개념을 소개하는 도입부
내용: # 머신러닝 소개
```

```
머신러닝은 데이터에서 패턴을 학습하는 AI의 한 분야입니다...

[Chunk 1] 지도학습 (분류와 회귀)
분할 이유: 지도학습의 두 가지 주요 유형을 함께 다루는 섹션
내용: ## 지도학습

지도학습은 레이블이 있는 데이터로 학습합니다...

[Chunk 2] 비지도학습과 클러스터링
분할 이유: 레이블 없는 학습 방법에 대한 별도 주제
내용: ## 비지도학습

비지도학습은 레이블 없이 데이터의 숨겨진 패턴을 찾습니다...

[Chunk 3] 강화학습
분할 이유: 완전히 다른 학습 패러다임으로 별도 분리
내용: ## 강화학습

에이전트가 환경과 상호작용하며 보상을 최대화하는 방향으로...
```

---

8. 문서 타입별 청킹

8.1 마크다운/HTML

```python
import re
from typing import List

class MarkdownChunker:
    """마크다운 문서 청커"""

    def __init__(self,
                 chunk_size: int = 500,
                 respect_headers: bool = True):
        self.chunk_size = chunk_size
        self.respect_headers = respect_headers

    def chunk(self, text: str) -> List[Chunk]:
        """헤딩 구조를 존중하며 청킹"""
        chunks = []

        if self.respect_headers:
            # 헤딩으로 분할
            sections = self._split_by_headers(text)

            for section in sections:
                if len(section["content"]) <= self.chunk_size:
                    chunks.append(Chunk(
                        text=section["content"],
                        index=len(chunks),
```

```
                        start_char=0,
                        end_char=len(section["content"]),
                        metadata={
                            "header": section["header"],
                            "level": section["level"]
                        }
                    ))
                else:
                    # 섹션이 너무 크면 추가 분할
                    sub_chunks = self._split_large_section(section)
                    chunks.extend(sub_chunks)

        return chunks

    def _split_by_headers(self, text: str) -> List[dict]:
        """헤딩으로 섹션 분할"""
        # 마크다운 헤딩 패턴
        header_pattern = r'^(#{1,6})\s+(.+)$'

        sections = []
        current_section = {"header": "", "level": 0, "content": ""}

        for line in text.split("\n"):
            match = re.match(header_pattern, line)
            if match:
                # 이전 섹션 저장
                if current_section["content"].strip():
                    sections.append(current_section)

                # 새 섹션 시작
                level = len(match.group(1))
                header = match.group(2)
                current_section = {
                    "header": header,
                    "level": level,
                    "content": line + "\n"
                }
            else:
                current_section["content"] += line + "\n"

        # 마지막 섹션 저장
        if current_section["content"].strip():
            sections.append(current_section)

        return sections

    def _split_large_section(self, section: dict) -> List[Chunk]:
        """큰 섹션을 작은 청크로 분할"""
        chunks = []
        content = section["content"]

        # 문단 단위로 분할
        paragraphs = content.split("\n\n")
        current_chunk = ""
```

```
        for para in paragraphs:
            if len(current_chunk) + len(para) <= self.chunk_size:
                current_chunk += para + "\n\n"
            else:
                if current_chunk:
                    chunks.append(Chunk(
                        text=current_chunk.strip(),
                        index=len(chunks),
                        start_char=0,
                        end_char=len(current_chunk),
                        metadata={
                            "header": section["header"],
                            "level": section["level"],
                            "is_split": True
                        }
                    ))
                current_chunk = para + "\n\n"

        # 마지막 청크
        if current_chunk.strip():
            chunks.append(Chunk(
                text=current_chunk.strip(),
                index=len(chunks),
                start_char=0,
                end_char=len(current_chunk),
                metadata={
                    "header": section["header"],
                    "level": section["level"],
                    "is_split": True
                }
            ))

        return chunks
```

8.2 코드

```
import ast
import re
from typing import List

class CodeChunker:
    """코드 파일 청커"""

    def __init__(self,
                 max_chunk_size: int = 1000,
                 include_context: bool = True):
        self.max_chunk_size = max_chunk_size
        self.include_context = include_context

    def chunk_python(self, code: str) -> List[Chunk]:
```

```python
        """Python 코드 청킹 (함수/클래스 단위)"""
        chunks = []

        try:
            tree = ast.parse(code)
        except SyntaxError:
            # 파싱 실패 시 라인 기반 분할
            return self._chunk_by_lines(code)

        # 클래스와 함수 추출
        for node in ast.walk(tree):
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef,
ast.ClassDef)):
                # 코드 범위 추출
                start_line = node.lineno - 1
                end_line = node.end_lineno

                code_lines = code.split("\n")
                chunk_code = "\n".join(code_lines[start_line:end_line])

                # 컨텍스트 추가 (import, 상위 클래스 등)
                if self.include_context:
                    context = self._get_context(code, start_line)
                    chunk_code = context + "\n\n" + chunk_code

                chunks.append(Chunk(
                    text=chunk_code,
                    index=len(chunks),
                    start_char=start_line,
                    end_char=end_line,
                    metadata={
                        "type": type(node).__name__,
                        "name": node.name,
                        "start_line": start_line + 1,
                        "end_line": end_line
                    }
                ))

        return chunks

    def _get_context(self, code: str, target_line: int) -> str:
        """코드의 상단 컨텍스트 추출 (import, 전역 변수 등)"""
        lines = code.split("\n")
        context_lines = []

        for i, line in enumerate(lines[:target_line]):
            if line.startswith("import ") or line.startswith("from "):
                context_lines.append(line)
            elif line.startswith("# ") and i < 5:  # 상단 주석
                context_lines.append(line)

        return "\n".join(context_lines)

    def _chunk_by_lines(self, code: str) -> List[Chunk]:
```

```
        """라인 기반 분할 (파싱 실패 시 대체)"""
        chunks = []
        lines = code.split("\n")
        current_chunk = []
        current_size = 0

        for i, line in enumerate(lines):
            if current_size + len(line) > self.max_chunk_size and current_chunk:
                chunks.append(Chunk(
                    text="\n".join(current_chunk),
                    index=len(chunks),
                    start_char=0,
                    end_char=0,
                    metadata={"type": "line_based"}
                ))
                current_chunk = []
                current_size = 0

            current_chunk.append(line)
            current_size += len(line) + 1

        if current_chunk:
            chunks.append(Chunk(
                text="\n".join(current_chunk),
                index=len(chunks),
                start_char=0,
                end_char=0,
                metadata={"type": "line_based"}
            ))

        return chunks


# 사용 예시
code = """
import numpy as np
from typing import List

def calculate_mean(numbers: List[float]) -> float:
    \"\"\"평균 계산\"\"\"
    return np.mean(numbers)

def calculate_std(numbers: List[float]) -> float:
    \"\"\"표준편차 계산\"\"\"
    return np.std(numbers)

class Statistics:
    \"\"\"통계 클래스\"\"\"

    def __init__(self, data: List[float]):
        self.data = data

    def summary(self) -> dict:
        return {
```

```
            "mean": calculate_mean(self.data),
            "std": calculate_std(self.data)
        }
"""

chunker = CodeChunker()
chunks = chunker.chunk_python(code)

print("=== Code Chunking 결과 ===")
for chunk in chunks:
    print(f"\n[{chunk.metadata['type']}] {chunk.metadata.get('name', 'N/A')}")
    print(f"라인: {chunk.metadata.get('start_line')}-
{chunk.metadata.get('end_line')}")
    print(f"내용:\n{chunk.text[:200]}...")
```

8.3 테이블 데이터

```
import pandas as pd
from typing import List

class TableChunker:
    """테이블/CSV 데이터 청커"""

    def __init__(self,
                 rows_per_chunk: int = 50,
                 include_header: bool = True):
        self.rows_per_chunk = rows_per_chunk
        self.include_header = include_header

    def chunk_dataframe(self, df: pd.DataFrame) -> List[Chunk]:
        """DataFrame을 청크로 분할"""
        chunks = []
        total_rows = len(df)

        # 헤더 생성
        header = " | ".join(df.columns.tolist())

        for start in range(0, total_rows, self.rows_per_chunk):
            end = min(start + self.rows_per_chunk, total_rows)
            chunk_df = df.iloc[start:end]

            # 테이블을 텍스트로 변환
            rows_text = []
            for _, row in chunk_df.iterrows():
                row_text = " | ".join(str(v) for v in row.values)
                rows_text.append(row_text)

            if self.include_header:
                chunk_text = f"| {header} |\n"
                chunk_text += "| " + " | ".join(["---"] * len(df.columns)) + "
|\n"
```

```
                chunk_text += "\n".join(f"| {r} |" for r in rows_text)
            else:
                chunk_text = "\n".join(rows_text)

            chunks.append(Chunk(
                text=chunk_text,
                index=len(chunks),
                start_char=start,
                end_char=end,
                metadata={
                    "row_start": start,
                    "row_end": end,
                    "columns": df.columns.tolist(),
                    "type": "table"
                }
            ))

        return chunks

    def chunk_csv_file(self, file_path: str) -> List[Chunk]:
        """CSV 파일 청킹"""
        df = pd.read_csv(file_path)
        return self.chunk_dataframe(df)
```

---

9. 실제 입출력 예시

9.1 텍스트 → 청크 → 임베딩 전체 과정

```
from sentence_transformers import SentenceTransformer
import numpy as np

# 샘플 텍스트
sample_text = """
삼성전자가 2024년 1분기 실적을 발표했습니다. 매출은 71조원으로 전년 동기 대비 5% 증가했
습니다.
반도체 부문이 회복세를 보이며 영업이익이 크게 개선되었습니다.

특히 HBM(고대역폭메모리) 수요가 급증하면서 메모리 사업부의 실적이 호조를 보였습니다.
AI 반도체 시장의 성장으로 HBM3 제품의 출하량이 전 분기 대비 50% 이상 증가했습니다.

모바일 부문에서는 Galaxy S24 시리즈가 출시 초기 호평을 받으며 판매량이 증가했습니다.
특히 AI 기능을 탑재한 것이 소비자들에게 좋은 반응을 얻었습니다.
"""

# 1. 청킹
chunker = RecursiveChunker(chunk_size=200, chunk_overlap=30)
chunks = chunker.chunk(sample_text)

print("=== 1. 청킹 결과 ===")
for chunk in chunks:
```

```
    print(f"\n[Chunk {chunk.index}] 길이: {len(chunk.text)}")
    print(f"텍스트: {chunk.text}")

# 2. 임베딩 생성
model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

print("\n=== 2. 임베딩 생성 ===")
for chunk in chunks:
    embedding = model.encode(chunk.text)
    print(f"\n[Chunk {chunk.index}]")
    print(f"임베딩 차원: {embedding.shape}")
    print(f"임베딩 값 (처음 10개): {embedding[:10]}")
    print(f"임베딩 노름: {np.linalg.norm(embedding):.4f}")
```

실제 출력:

```
=== 1. 청킹 결과 ===

[Chunk 0] 길이: 142
텍스트:
삼성전자가 2024년 1분기 실적을 발표했습니다. 매출은 71조원으로 전년 동기 대비 5% 증가했
습니다.
반도체 부문이 회복세를 보이며 영업이익이 크게 개선되었습니다.

[Chunk 1] 길이: 168
텍스트: 이익이 크게 개선되었습니다.

특히 HBM(고대역폭메모리) 수요가 급증하면서 메모리 사업부의 실적이 호조를 보였습니다.
AI 반도체 시장의 성장으로 HBM3 제품의 출하량이 전 분기 대비 50% 이상 증가했습니다.

[Chunk 2] 길이: 152
텍스트: 50% 이상 증가했습니다.

모바일 부문에서는 Galaxy S24 시리즈가 출시 초기 호평을 받으며 판매량이 증가했습니다.
특히 AI 기능을 탑재한 것이 소비자들에게 좋은 반응을 얻었습니다.

=== 2. 임베딩 생성 ===

[Chunk 0]
임베딩 차원: (384,)
임베딩 값 (처음 10개): [ 0.0234 -0.0567  0.1023 -0.0891  0.0456  0.0789 -0.0234
0.1123 -0.0456  0.0678]
임베딩 노름: 1.0000

[Chunk 1]
임베딩 차원: (384,)
임베딩 값 (처음 10개): [ 0.0456 -0.0234  0.0891 -0.1234  0.0567  0.0345 -0.0789
0.0912 -0.0123  0.0890]
임베딩 노름: 1.0000

[Chunk 2]
임베딩 차원: (384,)
```

```
임베딩 값 (처음 10개): [ 0.0123 -0.0890  0.0678 -0.0567  0.0891  0.0234 -0.0456
0.1012 -0.0345  0.0789]
임베딩 노름: 1.0000
```

9.2 청크 유사도 분석

```python
from sklearn.metrics.pairwise import cosine_similarity

# 청크 임베딩 생성
embeddings = model.encode([chunk.text for chunk in chunks])

# 유사도 행렬 계산
similarity_matrix = cosine_similarity(embeddings)

print("=== 청크 간 유사도 행렬 ===")
print("        Chunk0  Chunk1  Chunk2")
for i in range(len(chunks)):
    row = f"Chunk{i}  "
    for j in range(len(chunks)):
        row += f"{similarity_matrix[i][j]:.3f}   "
    print(row)

# 쿼리와의 유사도
query = "삼성전자 반도체 실적"
query_embedding = model.encode(query)

print(f"\n=== 쿼리 '{query}'와의 유사도 ===")
for i, chunk in enumerate(chunks):
    similarity = cosine_similarity([query_embedding], [embeddings[i]])[0][0]
    print(f"Chunk {i}: {similarity:.4f}")
    if similarity > 0.5:
        print(f"  → 관련 내용: {chunk.text[:50]}...")
```

실제 출력:

```
=== 청크 간 유사도 행렬 ===
        Chunk0  Chunk1  Chunk2
Chunk0  1.000   0.723   0.456
Chunk1  0.723   1.000   0.512
Chunk2  0.456   0.512   1.000

=== 쿼리 '삼성전자 반도체 실적'와의 유사도 ===
Chunk 0: 0.8234
  → 관련 내용: 삼성전자가 2024년 1분기 실적을 발표했습니다...
Chunk 1: 0.7891
  → 관련 내용: 이익이 크게 개선되었습니다. 특히 HBM(고대역폭메모리)...
Chunk 2: 0.4123
```

10. 청킹 최적화

10.1 성능 최적화

```
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor
from typing import List
import hashlib

class OptimizedChunker:
    """최적화된 청커"""

    def __init__(self, base_chunker, cache_dir: str = "./chunk_cache"):
        self.base_chunker = base_chunker
        self.cache_dir = cache_dir
        self._cache = {}

    def chunk_with_cache(self, text: str, doc_id: str = None) -> List[Chunk]:
        """캐시를 활용한 청킹"""
        # 해시 기반 캐시 키
        cache_key = doc_id or hashlib.md5(text.encode()).hexdigest()

        if cache_key in self._cache:
            return self._cache[cache_key]

        chunks = self.base_chunker.chunk(text)
        self._cache[cache_key] = chunks

        return chunks

    def chunk_batch(self,
                    texts: List[str],
                    max_workers: int = 4) -> List[List[Chunk]]:
        """배치 청킹 (병렬 처리)"""
        with ThreadPoolExecutor(max_workers=max_workers) as executor:
            results = list(executor.map(self.base_chunker.chunk, texts))
        return results

    def chunk_streaming(self, text_iterator):
        """스트리밍 청킹"""
        for text in text_iterator:
            yield self.base_chunker.chunk(text)
```

10.2 청크 품질 검증

```
from typing import List
from dataclasses import dataclass

@dataclass
class ChunkQualityMetrics:
    """청크 품질 메트릭"""
```

```
    avg_chunk_size: float
    size_variance: float
    overlap_ratio: float
    completeness_score: float  # 완결된 문장 비율
    coherence_score: float     # 의미적 일관성

class ChunkQualityChecker:
    """청크 품질 검증기"""

    def __init__(self,
                 min_chunk_size: int = 50,
                 max_chunk_size: int = 2000):
        self.min_chunk_size = min_chunk_size
        self.max_chunk_size = max_chunk_size

    def check_quality(self, chunks: List[Chunk]) -> ChunkQualityMetrics:
        """청크 품질 검사"""
        if not chunks:
            return ChunkQualityMetrics(0, 0, 0, 0, 0)

        # 크기 분석
        sizes = [len(c.text) for c in chunks]
        avg_size = sum(sizes) / len(sizes)
        variance = sum((s - avg_size) ** 2 for s in sizes) / len(sizes)

        # 오버랩 분석
        overlap_count = 0
        for i in range(1, len(chunks)):
            if chunks[i-1].text[-50:] in chunks[i].text[:100]:
                overlap_count += 1
        overlap_ratio = overlap_count / (len(chunks) - 1) if len(chunks) > 1 else
0

        # 완결성 검사 (마침표로 끝나는 비율)
        complete_count = sum(1 for c in chunks if c.text.strip().endswith(('.',
'!', '?', '。')))
        completeness = complete_count / len(chunks)

        return ChunkQualityMetrics(
            avg_chunk_size=avg_size,
            size_variance=variance,
            overlap_ratio=overlap_ratio,
            completeness_score=completeness,
            coherence_score=0.0  # 별도 임베딩 분석 필요
        )

    def validate(self, chunks: List[Chunk]) -> List[dict]:
        """청크 유효성 검증"""
        issues = []

        for chunk in chunks:
            chunk_issues = []

            # 크기 검사
```

```
            if len(chunk.text) < self.min_chunk_size:
                chunk_issues.append(f"너무 작음: {len(chunk.text)} <
{self.min_chunk_size}")
            if len(chunk.text) > self.max_chunk_size:
                chunk_issues.append(f"너무 큼: {len(chunk.text)} >
{self.max_chunk_size}")

            # 내용 검사
            if not chunk.text.strip():
                chunk_issues.append("빈 청크")
            if chunk.text.count('\n') / (len(chunk.text) + 1) > 0.3:
                chunk_issues.append("줄바꿈이 너무 많음")

            if chunk_issues:
                issues.append({
                    "chunk_index": chunk.index,
                    "issues": chunk_issues
                })

        return issues


# 사용 예시
checker = ChunkQualityChecker()
quality = checker.check_quality(chunks)
print(f"평균 청크 크기: {quality.avg_chunk_size:.0f}")
print(f"크기 분산: {quality.size_variance:.0f}")
print(f"완결성 점수: {quality.completeness_score:.2f}")

issues = checker.validate(chunks)
if issues:
    print("\n품질 이슈:")
    for issue in issues:
        print(f"  Chunk {issue['chunk_index']}: {issue['issues']}")
```

---

11. 평가 및 벤치마크

11.1 청킹 전략 비교 평가

```
from typing import List, Dict
import time

class ChunkingBenchmark:
    """청킹 전략 벤치마크"""

    def __init__(self, test_texts: List[str], ground_truth_chunks: List[List[str]]
= None):
        self.test_texts = test_texts
        self.ground_truth = ground_truth_chunks
```

```python
    def benchmark(self, chunkers: Dict[str, object]) -> Dict:
        """여러 청킹 전략 비교"""
        results = {}

        for name, chunker in chunkers.items():
            print(f"\n벤치마킹: {name}")

            # 속도 측정
            start_time = time.time()
            all_chunks = []
            for text in self.test_texts:
                chunks = chunker.chunk(text)
                all_chunks.append(chunks)
            elapsed = time.time() - start_time

            # 통계
            total_chunks = sum(len(c) for c in all_chunks)
            avg_chunks_per_doc = total_chunks / len(self.test_texts)
            avg_chunk_size = sum(
                len(chunk.text)
                for chunks in all_chunks
                for chunk in chunks
            ) / total_chunks if total_chunks > 0 else 0

            results[name] = {
                "total_time_seconds": elapsed,
                "docs_per_second": len(self.test_texts) / elapsed,
                "total_chunks": total_chunks,
                "avg_chunks_per_doc": avg_chunks_per_doc,
                "avg_chunk_size": avg_chunk_size
            }

        return results

    def print_results(self, results: Dict):
        """결과 출력"""
        print("\n" + "=" * 60)
        print("청킹 벤치마크 결과")
        print("=" * 60)

        header = f"{'전략':<20} {'시간(초)':<10} {'문서/초':<10} {'총 청크':<10}
{'평균크기':<10}"
        print(header)
        print("-" * 60)

        for name, metrics in results.items():
            row = f"{name:<20} {metrics['total_time_seconds']:<10.3f}
{metrics['docs_per_second']:<10.1f} {metrics['total_chunks']:<10}
{metrics['avg_chunk_size']:<10.0f}"
            print(row)


# 사용 예시
test_texts = [sample_text] * 100  # 100개 문서
```

```
chunkers = {
    "Fixed-500": FixedSizeChunker(chunk_size=500),
    "Recursive-500": RecursiveChunker(chunk_size=500),
    # "Semantic": SemanticChunker(threshold=0.5),  # 느림
}

benchmark = ChunkingBenchmark(test_texts)
results = benchmark.benchmark(chunkers)
benchmark.print_results(results)
```

예상 출력:

```
============================================================
청킹 벤치마크 결과
============================================================
전략                 시간(초)    문서/초    총 청크    평균크기
------------------------------------------------------------
Fixed-500            0.012      8333.3    300       487
Recursive-500        0.025      4000.0    350       421
```

---

12. 참고 자료

12.1 공식 문서

- LangChain Text Splitters
- LlamaIndex Node Parsers
- Chonkie - 청킹 라이브러리
- Jina AI Late Chunking

12.2 논문

- Chunking 2.0: A Smarter Way to Chunk Documents
- Beyond Fixed-Size Chunking

12.3 도구

- tiktoken - OpenAI 토크나이저
- sentence-transformers - 문장 임베딩
- spaCy - NLP 파이프라인